NeuroPAC: A Pre-Processing, Analysis, and Classification Pipeline Platform for EEG Signals

Selin Uygun

Supervisor: Assoc Prof Emine Elif Tülay

A Thesis Submitted in Partial Fulfillment of the Requirements

for the Degree of Bachelor of Science in Software Engineering

Department of Software Engineering

Mugla Sitki Kocman University

Abstract

(Abstract text goes here...)

Contents

Abstract **3**

List of Tables **8**

List of Figures **9**

I Introduction **10**

I.1 Background and Motivation 10

I.2 Problem Statement 11

I.3 Purpose of the Study 12

II Methodology **13**

II.1 Preprocessing Module 13

II.1.1 Preprocessing 13

II.1.2 Independent Component Analysis (ICA) & Artifact Removal 15

II.1.3 Event/Condition Decomposition 15

II.2 Analysis Module 15

II.3 Common Procedures During Analyses 16

II.3.1 File Management and Error Handling 16

II.3.2 FieldTrip Initialization 16

II.3.3 Batch Processing for Memory Efficiency 17

II.3.4 Baseline Correction 17

II.3.5 Result Storage and Output 17

II.3.6 ERP Analysis 18

Configuration and Computation 18
Baseline Correction and Output 18
II.3.7 Time-Frequency Analysis 18
Data Structure Normalization 19
Wavelet Configuration and Computation 19
Baseline Correction and Output 19
II.3.8 Spectral Analysis 19
Data Structure Normalization and Configuration 20
Computation and Output 20
II.3.9 Channel-Wise Connectivity Analysis 20
Data Normalization and Three-Stage Pipeline 21
Coherence Computation 21
Output 21
II.3.10 Inter-Trial Coherence Analysis 22
Data Normalization and Configuration 22
Two-Stage Computational Pipeline 22
Distinction from Channel-Wise Coherence and Output 23
II.4 Classification Module 23
II.4.1 EEGNet Classifier 23
Configuration Parameters 23
Architecture Overview 24
Design Rationale 25
II.4.2 EEG-Inception Classifier 25
Base Model Configuration 25
Architecture Overview 26
Time-Frequency Variant 26
Inter-Trial Coherence Variant 27
Architectural Variants 27
Design Philosophy 28

II.4.3 Riemannian Classifier 28
Theoretical Foundation 28
Base Pipeline Configuration 29
Pipeline Stages 29
Connectivity-Focused Implementation 30
Advantages and Limitations 31
II.5 Other Application Components 32
II.5.1 Template Items 32
Range Slider Template 32
Step Range Slider Template 32
Dropdown Menu Template 33
Input Box Template 33
Analysis Module Template 33
Classifier Template 35
II.5.2 MATLAB Parameter Parser & Dynamic Parameter Loader 40
MATLAB Parameter Parser 40
Dynamic Parameter Loader 40
II.5.3 Matlab Executor 41
Architectural Foundation and Core Responsibilities 41
Asynchronous MATLAB Execution Framework 42
Configuration Persistence and File Manipulation 43
Dynamic Parameter Reading and Interface Initialization 44
Path Resolution and Cross-Platform Compatibility 45
Custom Component Management 46
Module-Specific Targeting and Script Selection 47
Data Formatting and MATLAB Syntax Generation 48
Classification Pipeline Integration 49
Error Handling and User Feedback 50
State Management and Configuration Synchronization 51

II.5.4 File Browser Window . . . 52
II.5.5 Preprocessing-Specific . . . 55
ICA Browser . . . 55
Reject IC Button & `ft_databrowser` Synchronization . . . 56
II.5.6 Analysis-Specific . . . 59
Analysis Visualizer . . . 59
II.5.7 Classification-Specific . . . 61
Preprocess Bridge . . . 61
Controller . . . 63
Console . . . 63
Inference . . . 63
Main . . . 63
Model . . . 63
Test . . . 63
II.6 Research Design and Procedures . . . 63
II.7 Data Description . . . 63
II.7.1 Parkinson's . . . 63
II.8 Instrumentation . . . 63
II.9 Limitations . . . 63
II.10 Summary . . . 63

III Results **64**

IV Conclusion and Future Work **65**

IV.1 Conclusion . . . 65
IV.2 Future Work . . . 65

References **65**

Appendix A Assessment Activity **66**

Appendix B Assessment Rubric **67**

List of Tables

List of Figures

Chapter I

Introduction

I.1 Background and Motivation

Electroencephalography (EEG) is a non-invasive technique used to record electrical activity in the brain. It has become an essential tool in neuroscience research and clinical diagnostics due to its high temporal resolution, non-invasive nature, and relatively low cost. EEG signals are widely used to study brain function, cognitive processes, and neurological disorders such as epilepsy, Parkinson's disease, and Alzheimer's disease. EEG can help to diagnose certain conditions such as Alzheimer's disease by identifying characteristic patterns of brain activity associated with the disease. For example, EEG can detect changes in brain wave patterns that are indicative of cognitive decline, memory impairment, and brain disconnectivity. Additionally, EEG can be used to monitor the progression of neurodegenerative diseases and evaluate the effectiveness of treatments. By analyzing EEG data, researchers and clinicians can gain insights into the underlying mechanisms of neurodegenerative diseases and develop more effective diagnostic and treatment strategies.

3-Auditory Oddball Task is a widely used experimental paradigm in cognitive neuroscience to investigate attention, perception, and cognitive processing. In this task, participants are presented with a series of auditory stimuli, typically consisting of frequent standard tones, infrequent target tones and novel tones. The participants are instructed to respond to the target tones while ignoring the standard tones and aren't instruced on novel tones because they are unexpected and meant to capture attention involuntarily. The oddball task is designed to elicit specific event-related potentials (ERPs) in the EEG signal, such as the P300 component,

which is associated with attentional processes and cognitive evaluation of stimuli. The oddball task has been extensively used to study various cognitive functions, including attention, memory, and decision-making. It has also been employed in clinical research to investigate cognitive deficits in neurological and psychiatric disorders, such as schizophrenia, ADHD, and Alzheimer's disease.

I.2 Problem Statement

The core difficulty in Electroencephalography (EEG) data analysis lies in the trade-off between power/flexibility and usability/accessibility. Here are some limitations of two existing and most popular state of the art tools that are used for EEG pre-processing and analysis:

- **Fieldtrip, Complexity of Code-Based Toolboxes:** A high-performance, flexible toolbox like FieldTrip is essential for advanced analysis but rely on MATLAB code and scripting. This necessitates a high level of programming expertise, creating a significant barrier to entry and slowing down the research process for those whose primary focus is neuroscience or clinical application rather than coding and data structuring.

- **EEGLAB, Existing GUI Complexity:** While EEGLAB offers a Graphical User Interface (GUI), this interface can present a dense, overwhelming array of parameters, menus, and advanced options to a novice. This complexity often fails to make the fundamental, necessary workflow—preprocessing, processing, and classification—feel less intimidating or more welcoming to beginners.

There is a clear and persistent need for a dedicated, simplified interface that explicitly targets the essential and initial steps of EEG analysis, thereby democratizing the process.

NeuroPAC addresses the need for a streamlined, intuitive Graphical User Interface (GUI) designed to guide users through the crucial, initial steps of preprocessing, processing, and deep learning classification in a less intimidating manner. By building on the powerful computational back-end of toolboxes like FieldTrip, this GUI will abstract away the code, presenting only the core configuration options needed for a reproducible, foundational analysis. The efficacy of

this simplified pipeline must be demonstrated in a real-world, high-stakes context. There is a need to validate this new streamlined analysis tool by successfully applying it to challenging neurological datasets, specifically in generating robust and clinically meaningful deep learning classification results for conditions such as Parkinson's Disease.

In summary, the basic difficulty is the lack of a user-friendly tool that simplifies the essential EEG analysis pipeline (preprocessing, feature extraction analysis, and classification). Existing tools are either highly code-dependent (FieldTrip) or still too complex for beginners (EEGLAB). The perceived need is thus the development of a simplified, welcoming user interface that leverages existing powerful toolboxes to make foundational EEG analysis accessible, efficient, and reproducible, with its utility confirmed through successful application and validation in classifying clinical cohorts like Parkinson's Disease patients.

I.3 Purpose of the Study

The purpose of this study is two-fold: to address the usability gap in existing neuroscientific software and to validate the resulting platform through compelling clinical application.

- **Engineering Goal (Product):** To develop a user-centric Graphical User Interface (GUI) using Python that provides a streamlined, accessible, and intuitive workflow for the fundamental steps of EEG data analysis (preprocessing, processing, and classification). This GUI will abstract the complexities of code-based toolboxes like FieldTrip, thereby lowering the technical barrier to entry and welcoming new users to the field of neuroscience and EEG analysis.
- **Research Goal (Practical Outcomes):** To validate the utility and efficacy of the developed GUI and its integrated deep learning classification pipeline by generating meaningful, publishable findings. This includes providing example EEG data outcomes, specifically robust analysis results and accurate classification metrics for clinical datasets concerning Parkinson's Disease.

The ultimate goal is to deliver a practical product that enhances the efficiency of the EEG analysis pipeline and produces high-quality, reproducible research outcomes.

Chapter II

Methodology

II.1 Preprocessing Module

II.1.1 Preprocessing

Preprocessing is the very first step of the EEG analysis pipeline. It includes adjusting the raw EEG data to make it suitable for further analysis. To analyze a dataset, we must first initialize a cfg data structure from the FieldTrip toolbox, which serves as our data container and provides numerous configurable properties. The preprocessing module of NeuroPAC streamlines this process by allowing users to set cfg structure properties directly through the GUI, automating the workflow without requiring manual code configuration. The preprocessing steps currently implemented in NeuroPAC as base preprocessing components are as follows:

- `cfg.trialfun = 'ft_trialfun_general'` : Defines trials based on specified events and time windows.
- `cfg.trialdef.eventtype = 'stimulus'`: Specifies the type of event to use for trial definition.
- `cfg.trialdef.eventvalue = {'S200', 'S201', 'S202'}` : Sets the specific event value to identify trials (conditions).
- `cfg.channel = 'all'` : Selects all available EEG channels for analysis.
- `cfg.demean = 'yes'` : Removes the mean value from each channel.

- `cfg.baselinewindow = [-0.2 0]` : Defines the time window for baseline correction.
- `cfg.dftfilter = 'yes'` : Enables the use of a notch filter to remove line noise.
- `cfg.dftfreq = [50 60]` : Specifies the frequencies to be removed by the notch filter (e.g., 50 Hz and its harmonics).

These steps can be deleted, modified, or new ones can be added from the top menu as needed. These are individual lines that get converted from simple MATLAB script lines to specific QML template items by the parameter parser, you can read further about the parameter parser and template items themselves on sections 3.3.2 and 3.3.3. Before moving on to running preprocessing, the user has to make sure that:

- Their fieldtrip path is correctly set from the top menu,
- and the file browser is pointing to the correct directory where the raw EEG data is located.

After these configurations are set, the user can run the `Preprocess and Run ICA` button, which executes a Fieldtrip function called `ft_preprocessing` that applies the defined preprocessing steps to the raw EEG data. This operation will require to run on MATLAB, therefore the MATLAB will open to ensure a smooth execution and will provide any warnings from its console that the user might need to be aware of. After the preprocessing is done, the preprocessed data will be saved in a file named `data.mat` in the same directory as the raw data. After preprocessing is done, the button will fulfill its second purpose, which is to run ICA on the preprocessed data. ICA is a computational dimension reduction method used to separate a multivariate signal into additive, independent components. In the context of EEG data, ICA is commonly employed to identify and remove artifacts (unwanted signals) such as eye blinks, muscle activity, and other noise sources that can contaminate the EEG recordings. In our case ICA is chosen as `fast_ica` algorithm provided by Fieldtrip, and in this study's case only the eye artifacts were considered and removed. After running ICA, the resulting components will be saved in a file named `data_ICApplied.mat` in the same directory as the raw data for further inspection and artifact removal.

II.1.2 Independent Component Analysis (ICA) & Artifact Removal

The user, if ready to proceed, can then move on to the artifact removal step by clicking the `data_ICApplied.mat` file name directly from the file browser, which will open a matlab window displaying a slightly modified version of `ft_databrowser` which now contains a "Reject ICA" button on the right of the browser window. If any user wants to replicate the same application experience, they would have to run the syncing script, which is given as an option in the top menu. This will help to modify their own "`ft_databrowser`" script by adding the "Reject IC" button. The independent components obtained from ICA will be shown and iterated for every subject. The user can then visually inspect these components and select the ones corresponding to artifacts (e.g., eye blinks) for removal with the "Reject IC" button. Once the user selects the components to be removed and confirms their choice, the array with the rejected IC's will be saved and then be iterated through the whole data, removing each rejected IC of the corresponding subject. The system will reconstruct the EEG data without the selected artifact components.

II.1.3 Event/Condition Decomposition

After the IC rejection, a quick event (trial type) decomposition is made to ready the final data for analyses. The decomposition's job is to decompose the trials by their corresponding numbers 200, 201, 202 to target, standard and novelty respectively. The outcome of this final process results in a subject number $\times$ 1 struct with three fields (three being the trial conditions), and then the cleaned and decomposed data is saved in a file named `data_ICApplied_clean_decomposed.mat` in the same currently open file browser directory. This cleaned and decomposed data is now ready for feature extraction and analysis in the Analysis Module.

II.2 Analysis Module

The analysis module of NeuroPAC provides a suite of commonly used EEG analysis techniques that can be easily configured and executed through the GUI. The module allows users to select from various analysis methods, set relevant parameters, and visualize the results. The available

analyses include:

- Event-Related Potential (ERP) Analysis,
- Time-Frequency Analysis,
- Spectral Analysis,
- Channel-Wise Connectivity Analysis,
- Inter-Trial Coherence Analysis.

These analyses are commonly used in EEG research and provide a solid foundation for understanding various aspects of EEG data. Each analysis serves a specific purpose and can be further customized or expanded, or can be further feature-extracted before the classification step based on the user's research needs.

II.3 Common Procedures During Analyses

All analysis modules in NeuroPAC follow a standardized procedural framework to ensure consistency, reliability, and efficient resource management across different analysis types. The key shared components are:

II.3.1 File Management and Error Handling

Analysis functions search for the standard preprocessing output file `data_ICApplied_clean_decomposed.ma` accepting either direct file paths or folder paths. The functions intelligently extract parent directories if needed and verify the existence of the expected `clean_data_decomposed` variable, providing clear error messages if data integrity issues arise. All file operations are wrapped in try-catch blocks for robust error handling.

II.3.2 FieldTrip Initialization

Each analysis script automatically detects and initializes FieldTrip if not already available on the MATLAB path. The detection routine first checks configuration files from the preprocess-

ing module for previously specified paths, then searches common installation directories. If FieldTrip is located, the script adds it to the path and calls `ft_defaults`. If initialization fails, the script provides clear error messages with manual setup instructions.

II.3.3 Batch Processing for Memory Efficiency

To handle large datasets without exhausting system memory, all analysis scripts process subjects in batches of ten. This approach allows analyses to complete successfully on systems with limited resources while maintaining computational efficiency. Console output marks the beginning and completion of each batch, and subjects within each batch are processed sequentially across all three experimental conditions (target, standard, novelty).

II.3.4 Baseline Correction

Following primary analysis computations, all applicable analyses apply baseline correction using FieldTrip's dedicated functions. The baseline correction normalizes measurements relative to pre-stimulus activity, ensuring observed responses reflect stimulus-induced changes rather than pre-existing offsets. Baseline parameters vary by analysis type but follow a consistent approach: defining a pre-stimulus window, computing mean values, and subtracting these from post-stimulus periods. Note that some analyses (spectral and connectivity-based) omit baseline correction when the measure itself provides normalized outputs.

II.3.5 Result Storage and Output

All analysis scripts save results in well-organized data structures arranged by subject and condition, including essential metadata (time vectors, frequency vectors, channel labels). Output files use descriptive naming conventions clearly indicating the analysis type and are saved alongside the source data. Throughout processing, scripts maintain clear console communication confirming completion stages and providing final output file locations.

II.3.6 ERP Analysis

Event-Related Potential (ERP) analysis examines brain responses time-locked to experimental stimuli. In NeuroPAC, the `timelock_func.m` script implements ERP analysis using FieldTrip's timelock analysis functions following the common procedural framework.

Configuration and Computation

The analysis configures a time window from 0 to 1 second post-stimulus using the parameter `cfg.latency`. FieldTrip's `ft_timelockanalysis` function averages individual trials within each condition to produce the characteristic ERP waveform, enhancing signal-to-noise ratio by reinforcing time-locked neural activity. The script processes target, standard, and novelty conditions conditionally, skipping empty data fields.

Baseline Correction and Output

Baseline correction uses a pre-stimulus window from -0.2 to 0 seconds, representing the 200 milliseconds immediately preceding stimulus onset. This short baseline period captures the immediate pre-stimulus state while avoiding contamination from earlier events. The correction employs absolute subtraction via FieldTrip's `ft_timelockbaseline` function.

Results are reformatted for compatibility with visualization modules, creating a subjects-by-conditions matrix containing time vectors, averaged voltage values, channel labels, variance measures, and degrees of freedom. The script saves two complementary structures: `ERP_data` (detailed timelock output) and `erp_records` (reformatted for visualization) in `erp_output.mat`.

II.3.7 Time-Frequency Analysis

Time-frequency analysis reveals how spectral content evolves over time, preserving both temporal and frequency dimensions simultaneously. This complements ERP analysis (time-domain) and spectral analysis (frequency-domain) by capturing dynamic changes in neural oscillations. In NeuroPAC, the `timefreqanalysis.m` script implements wavelet-based decomposition following the common procedural framework.

Data Structure Normalization

The script inspects and normalizes input data to handle varying organizational formats from different preprocessing workflows. Regardless of whether data uses field names like `target`, `target_data`, or matrix organization, the normalization produces a consistent structure where each subject contains three condition fields (target, standard, novelty), enabling uniform processing.

Wavelet Configuration and Computation

The analysis employs wavelet decomposition with the following configuration: frequency range from 1 to 15 Hz (half-hertz increments, covering delta/theta/alpha bands), time points from 0 to 1 second post-stimulus (10-millisecond steps), wavelet width of 3 cycles, and 8-second padding to prevent edge artifacts. FieldTrip's `ft_freqanalysis` function computes power spectra at each time-frequency point using complex Morlet wavelets, capturing event-related synchronization and desynchronization patterns.

Baseline Correction and Output

Baseline correction uses a pre-stimulus window from -1 to -0.5 seconds, providing a longer baseline than ERP analysis for more stable power estimates across frequency bands. The correction applies absolute subtraction via FieldTrip's `ft_freqbaseline` function. Results are packaged into `timefreq_data` (organized by subject and condition, including power values, time vectors, frequency vectors, and channel labels) and saved to `timefreq_output.mat`.

II.3.8 Spectral Analysis

Spectral analysis examines frequency content averaged across entire trial epochs, complementing time-frequency analysis which preserves temporal dynamics. This approach characterizes baseline oscillatory properties and frequency-specific changes sustained throughout experimental conditions. In NeuroPAC, the `spectralanalysis.m` script implements Fourier-based decomposition.

Data Structure Normalization and Configuration

The script implements the same data structure inspection and normalization phase used in time-frequency analysis, handling various organizational formats and producing a consistent structure where each subject contains three condition fields (target, standard, novelty). The configuration specifies the multitaper fast Fourier transform method (`mtmfft`) with Fourier coefficients as output (preserving phase information for downstream connectivity analyses), Hanning taper windowing, and frequency range from 1 to 15 Hz (half-hertz increments, covering delta/theta/alpha bands).

Computation and Output

FieldTrip's `ft_freqanalysis` function applies the multitaper Fourier transform to all trials for each subject-condition combination with valid data, computing Fourier coefficients for each frequency bin, channel, and trial. Unlike ERP and time-frequency analyses, spectral analysis omits baseline correction because coherence inherently measures relative phase consistency and Fourier coefficients represent spectral composition without requiring baseline normalization. Results are packaged into `spectral_data` (organized by subject and condition, containing Fourier coefficients, frequency vectors, channel labels, and metadata) and saved to `spectral_output.mat`.

II.3.9 Channel-Wise Connectivity Analysis

Channel-wise connectivity analysis investigates functional relationships between different brain regions by quantifying how neural signals from different electrode locations interact with each other. Unlike signal-level analyses that examine individual channels in isolation, connectivity analysis characterizes the synchronization of neural oscillations across the scalp. In NeuroPAC, the `channelwise.m` script computes coherence measures between all pairs of EEG channels.

Data Normalization and Three-Stage Pipeline

The script implements data structure normalization to handle various organizational formats from preprocessing workflows. The analysis requires a three-stage computational pipeline: (1) frequency decomposition using wavelet analysis with Fourier coefficient output (preserving phase information), (2) temporal selection specifying a latency window from 0 to 1 second to exclude edge artifacts, and (3) coherence computation quantifying phase relationship consistency between channel pairs. The wavelet configuration mirrors time-frequency analysis (frequency range 1-15 Hz in half-hertz increments, 10-millisecond time steps, 3-cycle wavelet width, 8-second padding).

Coherence Computation

FieldTrip's `ft_freqanalysis` produces time-frequency representations with Fourier coefficients, then `ft_selectdata` extracts the specified latency window, and `ft_connectivityanalysis` computes coherence values between all channel pairs. Coherence (range 0-1) measures the consistency of phase relationships across trials; values near zero indicate weak functional coupling while values near one indicate strong phase synchronization. Unlike correlation-based measures, coherence examines frequency-resolved phase relationships, avoiding volume conduction artifacts.

Output

Coherence analysis omits baseline correction because coherence inherently measures relative phase consistency without reference to absolute signal amplitudes. Results are packaged into `coherence_data` (organized by subject and condition, containing coherence matrices for all channel pairs across frequencies, channel labels, and frequency vectors) and saved to `channelwise_coherence_output.mat`, enabling examination of both local and long-range connectivity patterns.

II.3.10 Inter-Trial Coherence Analysis

Inter-trial coherence analysis quantifies the extent to which neural oscillations are phase-locked to experimental events, examining phase consistency across trials at the same channel location rather than across different channels. This reveals whether oscillatory activity timing relative to stimulus onset is consistent across repeated presentations. High inter-trial coherence indicates stimulus-driven entrainment of neural oscillations, while low values indicate variable phase relationships. In NeuroPAC, the `intertrialcoherenceanalysis.m` script implements phase-locking quantification.

Data Normalization and Configuration

The script implements the same data structure inspection and normalization phase used in other frequency-domain analyses, handling various organizational formats with adaptive field-selection logic. The configuration parallels channel-wise connectivity analysis: wavelet-based frequency decomposition with Fourier coefficient output (preserving phase information), frequency range from 1 to 15 Hz (half-hertz increments), time points from 0 to 1 second (10-millisecond steps), wavelet width of 3 cycles, and 8-second padding. Fourier coefficients are essential because inter-trial coherence fundamentally depends on phase information.

Two-Stage Computational Pipeline

The analysis executes a two-stage pipeline for each subject-condition combination. First, FieldTrip's `ft_freqanalysis` produces a time-frequency representation with Fourier coefficients for each channel, frequency, time point, and trial. Second, a dedicated `compute_itc` function examines Fourier coefficients for all trials at each channel, frequency, and time point, computing the degree to which phase angles are aligned across the trial ensemble. The resulting inter-trial coherence values (range 0-1) indicate phase-locking strength; zero represents completely random phase relationships while one indicates perfect phase alignment across all trials.

Distinction from Channel-Wise Coherence and Output

Inter-trial coherence and channel-wise coherence measure fundamentally different phenomena. Channel-wise coherence quantifies phase synchronization between two spatial locations across trials; inter-trial coherence quantifies phase synchronization across trials at a single location. Channel-wise coherence reveals spatial organization of brain networks, while inter-trial coherence reveals temporal precision of stimulus-evoked responses. Like other frequency-domain analyses, inter-trial coherence omits baseline correction because the measure inherently quantifies relative phase consistency.

Results are packaged into two complementary structures: `itc_data` (standard structure array organized by subject and condition) and `itc_records` (cell array format for flexible downstream operations). Both are saved to `intertrial_coherence_output.mat`, providing organizational flexibility for different analytical needs.

II.4 Classification Module

II.4.1 EEGNet Classifier

EEGNet is a compact deep learning architecture designed specifically for EEG signal classification. Originally introduced by Lawhern et al. in 2018, the model addresses the unique challenges of EEG data by incorporating separate temporal and spatial filtering stages that mirror traditional signal processing approaches while remaining parameter-efficient for limited training data scenarios.

Configuration Parameters

Key parameters control the model architecture:

- `nb_classes`: Number of classification categories
- `Chans`, `Samples`: Number of EEG channels and time points per trial
- `F1`, `D`, `F2`: Filter counts for temporal, spatial, and separable convolution stages

- `kernLength`: Temporal extent of initial filters (recommended: half the sampling rate)
- `dropoutRate`, `norm_rate`: Regularization parameters controlling overfitting
- `dropoutType`: Choice between Dropout and SpatialDropout2D based on data type

Architecture Overview

The input tensor is organized as channels by time points by a singleton feature dimension, treating channels analogously to image height and time analogously to width. The architecture progresses through two primary convolutional blocks followed by a classification layer.

First Block: Temporal and Spatial Filtering The first block learns both temporal and spatial patterns. A temporal convolution with `F1` filters spans one channel by `kernLength` time points, detecting temporal patterns across all channels simultaneously. This is followed by batch normalization and a depthwise convolution that applies spatial filters of all channels by one time point, creating `F1 * D` feature maps. The `depthwise_constraint` regularization limits filter norms to one, preventing individual filters from dominating. An exponential linear unit (ELU) activation introduces nonlinearity. Average pooling reduces temporal resolution by a factor of four, and dropout prevents overfitting.

Second Block: Feature Combination The second block uses separable convolutions to optimally combine spatial features across the temporal dimension. Depthwise filters span one by sixteen time points, followed by a pointwise convolution with `F2` filters that learns weighted combinations of features. A more aggressive average pooling (kernel size one by eight) further reduces temporal resolution, creating abstract representations of longer-range dependencies.

Classification Layer A flattening operation concatenates all feature dimensions into a one-dimensional vector. A single fully connected dense layer maps this to the output space with softmax activation. The `kernel_constraint` parameter regularizes the classification weights.

Design Rationale

EEGNet's effectiveness derives from its careful architectural decisions. By separating temporal and spatial filtering, the model learns interpretable features aligned with neuroscientific understanding. Parameter efficiency enables effective training with limited data. The use of depthwise separable convolutions reduces computational cost while maintaining representational capacity. The constraint-based regularization (norm constraints on depthwise and classification layers) prevents overfitting by encouraging diverse, distributed feature representations rather than reliance on a few dominant filters.

II.4.2 EEG-Inception Classifier

EEG-Inception employs parallel multi-scale processing that simultaneously examines input data at different temporal resolutions, contrasting with EEGNet's sequential temporal-then-spatial approach. This architecture recognizes that neural signals contain information at multiple time scales, from fast transient responses to slower oscillatory patterns. NeuroPAC provides three variants: a base model for raw time series, a time-frequency variant for spectrograms, and an inter-trial coherence variant for phase consistency patterns.

Base Model Configuration

Key parameters control the architecture:

- `input_time`, `fs`: Epoch duration (ms) and sampling frequency; together determine temporal samples
- `ncha`: Number of EEG channels
- `filters_per_branch`: Filters per parallel processing branch
- `scales_time`: List of temporal scales in milliseconds (default: 500, 250, 125 ms)
- `dropout_rate`, `activation`, `n_classes`, `learning_rate`: Standard regularization and training parameters

Architecture Overview

The input tensor is organized as temporal samples by channels by a singleton feature dimension. The architecture progresses through multiple Inception blocks that progressively compress and refine multi-scale features.

First Inception Block Creates parallel processing pathways for each temporal scale specified. Each pathway applies: (1) temporal convolution spanning the full scale by one channel with He normal initialization, (2) batch normalization, (3) ELU activation, (4) depthwise convolution with depth multiplier of two and max-norm constraint of one, (5) batch normalization and dropout. All pathways are concatenated along the feature dimension, producing a unified multi-scale representation. Adaptive average pooling further reduces temporal resolution while preventing degenerate dimensions.

Second Inception Block Operates on pooled features from the first block using reduced kernel sizes (one-quarter of original scales). Excludes depthwise spatial convolutions, focusing instead on combining and abstracting spatially-filtered features from the first block. Concatenated outputs undergo adaptive pooling for further downsampling.

Classification Stages A sequential refinement pathway progressively reduces feature dimensionality through two convolutional stages with adaptive kernel sizes and filter counts that create a bottleneck, forcing the network to learn the most discriminative features. Flattening converts feature maps to a one-dimensional vector, and a dense layer with softmax produces class probabilities. Training uses Adam optimizer with specified learning rate and binary cross-entropy loss.

Time-Frequency Variant

Operates on spectrogram data (frequency bins $\times$ time points $\times$ channels) from wavelet decomposition. Parameters `n_freqs`, `n_times`, `n_channels` specify dimensions (defaults: 91, 29, 12). Multi-scale kernel sizes are defined in frequency (`scales_freq`: defaults 3, 5, 9) and time (`scales_time`: defaults 3, 5, 7) dimensions.

More aggressive regularization includes spatial dropout (0.15) after input and channel expansion via one-by-one convolutions. Inception blocks employ separable convolutions (decomposed into depthwise and pointwise) with 12 filters per branch. Classification consists of dense layers (128, 64 units) with batch normalization and dropout (0.4, 0.3).

A critical innovation uses focal loss instead of standard cross-entropy, emphasizing hard-to-classify examples. The modulating factor downweights well-classified examples, with focusing parameter $\gamma = 4.0$ creating strong emphasis on difficult cases. This proves valuable in medical diagnostic applications with class imbalance.

Inter-Trial Coherence Variant

Mirrors the time-frequency architecture but adds class-specific weighting via the `class_weights` parameter. The `calculate_class_weights` function automatically computes weights based on class frequency: *balanced* strategy applies inverse frequency weighting, while *aggressive* strategy amplifies minority class weights by raising ratios to power 1.5. Weights are normalized so the mean equals one. Focal loss integrates these weights, allowing examples from high-weight classes to influence training more strongly, handling severe class imbalance and varying diagnostic importance.

Architectural Variants

Both time-frequency and inter-trial coherence variants provide simplified alternatives:

- **V2 variants**: Implement residual connections (ResNet-inspired) with shortcut pathways facilitating gradient flow through deeper networks
- **Lightweight variants**: Shallower architectures with fewer layers and smaller filters for small datasets
- **Simple variants**: Minimal architectures with single convolutional blocks or direct classification, suitable when training data is extremely limited

Model complexity is matched to data availability, with simpler models preferred when data is scarce.

Design Philosophy

EEG-Inception's parallel multi-scale processing enables the network to learn which temporal scales matter most for a given task, rather than making predetermined assumptions. The availability of variants for different input formats (raw time series, spectrograms, phase consistency) reflects that EEG can be represented multiple ways, and optimal architectures must respect each format's statistical properties. Advanced training techniques (focal loss, class weighting) address practical challenges in medical diagnosis. The flexibility to match model complexity to data characteristics creates a framework capable of handling diverse EEG analysis scenarios while remaining grounded in neuroscientific principles.

II.4.3 Riemannian Classifier

The Riemannian classifier employs differential geometry and manifold theory to classify EEG data, contrasting with deep learning approaches. It recognizes that covariance matrices derived from multi-channel EEG do not exist in Euclidean space but on a curved Riemannian manifold (symmetric positive definite matrices). By employing geometric operations that preserve manifold structure, this approach often achieves superior classification performance. NeuroPAC provides two variants: a base implementation for covariance matrices and a connectivity-focused implementation for functional connectivity measures.

Theoretical Foundation

Covariance matrices satisfy two fundamental properties: symmetry and positive definiteness (all eigenvalues strictly positive). These constraints define a curved manifold rather than flat Euclidean space. Euclidean operations fail to respect this geometry; for example, arithmetic means of covariance matrices in Euclidean fashion may not be optimal on the manifold, and Euclidean distances misrepresent geometric relationships. The Riemannian framework provides mathematical tools to work directly on the manifold, defining appropriate metrics, geodesics, and means that respect intrinsic geometry.

Base Pipeline Configuration

Key parameters control the pipeline:

- `estimator`: Covariance estimation method (sample covariance, Ledoit-Wolf shrinkage, oracle approximating shrinkage, minimum covariance determinant)
- `metric`: Riemannian metric for geometric operations (affine-invariant `riemann`, logarithmic Euclidean `logeuclid`, standard `euclid`)
- `use_mdm`: Enable Minimum Distance to Mean classifier
- `use_deep`: Enable deep learning classification via multilayer perceptron
- `kernel`, `C`: SVM kernel type and regularization when using SVM classification
- Deep learning parameters: `hidden_dims`, `dropout`, `lr`, `max_epochs`, `batch_size`, `weight_decay`, `patience`

Pipeline Stages

Covariance Estimation The Covariances transformer accepts raw EEG data (trials $\times$ channels $\times$ time points) and produces covariance matrices (trials $\times$ channels $\times$ channels), computing second-order statistics characterizing spatial relationships. Shrinkage-based estimators (Ledoit-Wolf, oracle approximating shrinkage) provide stable estimates when channels approach or exceed temporal samples, preventing ill-conditioned matrices. Each covariance matrix represents a trial as a point on the manifold, encoding spatial correlation structure while discarding temporal dynamics.

Tangent Space Projection For classical or deep learning classification, the TangentSpace transformer linearizes curved manifold geometry into flat Euclidean space suitable for conventional algorithms. A reference point (typically the Riemannian mean of training covariance matrices) serves as the tangent space origin. The Riemannian metric computes this mean via iterative optimization minimizing squared geodesic distances; log-Euclidean metric uses closed-form solutions via matrix logarithms. Each covariance matrix maps to the tangent space via

logarithmic mapping, computing the tangent vector along the geodesic connecting reference and target. Resulting tangent vectors have dimensionality $n_{channels}(n_{channels}+1)/2$, creating Euclidean feature vectors that preserve local geometric relationships while enabling standard machine learning operations.

Classification Methods **Minimum Distance to Mean (MDM)**: Computes a Riemannian mean for each class, classifying new trials by geodesic distance to class means. This pure geometric approach respects manifold structure throughout, avoiding linearization approximations. Effective when classes exhibit distinct covariance patterns forming separated clusters on the manifold and training samples per class are sufficient for stable mean estimation. Higher computational cost due to eigenvalue decompositions and matrix logarithms compared to tangent space methods.

Deep Learning: The TFMLPClassifier receives tangent space features after standardization (zero mean, unit variance). Network architecture includes input dropout, hidden layers with dense transformations, ReLU activation, batch normalization, and dropout, with optional L2 weight decay. Training uses Adam optimizer and sparse categorical cross-entropy loss. Early stopping with validation monitoring prevents overfitting. This hybrid approach combines Riemannian feature extraction with neural network classification.

Support Vector Machine: Operates on standardized tangent space features, constructing maximum-margin hyperplanes. The `kernel` parameter determines transformations: linear kernel for linearly separable classes, radial basis function for complex nonlinear boundaries, polynomial kernels for polynomial decision surfaces. The `C` parameter controls margin-error trade-off. Class weighting handles imbalanced datasets with inverse frequency scaling. Probability estimation fits logistic regression on SVM decision values.

Connectivity-Focused Implementation

Functional connectivity matrices (coherence, phase locking value, correlation) may violate positive definiteness assumptions due to estimation noise or singular conditions, causing standard Riemannian methods to fail. The connectivity pipeline introduces regularization ensuring all matrices satisfy geometric requirements while preserving discriminative information.

The ConnectivityRegularizer transformer applies multi-stage regularization: (1) symmetry correction by averaging with transpose, (2) diagonal regularization adding small positive values to shift eigenvalues upward, (3) shrinkage toward identity providing numerical stability, (4) optional eigenvalue clipping explicitly enforcing positive definiteness. The `preserve_scale` option maintains matrix trace, preventing systematic magnitude distortion.

The SoftTangentSpace transformer extends standard tangent space projection with error handling and fallback mechanisms. Pre-regularization strengthens mean computation stability. Exception handling catches failures and falls back to log-Euclidean or Euclidean operations when full Riemannian approach encounters numerical difficulties, ensuring pipeline completion while noting fallback activation for data quality investigation.

Alternative vectorized mode bypasses Riemannian geometry, extracting the upper triangle of connectivity matrices as feature vectors. The ConnectivityVectorizer applies optional logarithmic transformation improving distributions when values span multiple orders of magnitude. This approach sacrifices geometric principles for simplicity and efficiency, proving effective when manifold geometry contributes minimally or matrices are severely corrupted. Deep learning classification operates on vectorized features with more aggressive regularization (dropout 0.75, smaller hidden layers, reduced batch size 8) appropriate for small connectivity datasets.

Advantages and Limitations

Advantages: Interpretability through covariance/connectivity patterns directly relating to functional brain relationships; parameter efficiency enabling effective classification with smaller training sets since covariance estimation dramatically reduces dimensionality; mathematical elegance through principled geometric handling.

Limitations: Loss of temporal information through static covariance matrices; stationarity assumptions implicit in covariance estimates; sensitivity to estimation quality, particularly problematic with short trials or high channel counts.

Flexibility to choose pure geometric methods (MDM, best with limited data) or hybrid approaches (combining Riemannian extraction with deep learning, advantageous with sufficient data) enables adaptation to dataset characteristics. The connectivity variant with regularization

extends applicability to practical scenarios where strict positive definiteness is violated.

II.5 Other Application Components

II.5.1 Template Items

Range Slider Template

The range slider QML component displays a pair of draggable handles to select a numeric interval, keeps text fields in sync for min/max/current values, and snaps changes to a configurable step with sensible rounding. It supports an edit/add mode that lets users rename the bound MATLAB property, preview the exact assignment text (including special handling for trial windows), and persist or delete the property via hooks to the global `matlabExecutor`. Input validation prevents inverted ranges, enforces bounds, and surfaces warnings inline; helper functions also update related MATLAB settings when specialized slider IDs (baseline, dftfreq, prestim/poststim, ERP) are used. Visual affordances include background highlighting of the active span, focus-driven z-ordering, and icon buttons for saving or removing the parameter while respecting read-only vs. edit states.

Step Range Slider Template

This stepped range slider renders a double-handled control for values defined in MATLAB's start:step:end syntax, keeps paired text inputs synchronized for bounds, step size, and handle positions, and snaps all changes to the current step with precision-aware rounding. An add/edit mode lets users rename the underlying `cfg` property, preview the exact colon-form assignment, and persist or delete it via `matlabExecutor` hooks; validation guards against reversed bounds, zero/negative steps, and out-of-range entries with inline warnings. When slider IDs match known preprocessing roles (baseline, dftfreq, prestim/poststim, ERP) it calls specialized executor updaters; otherwise it saves dynamic parameters through the step-range MATLAB writer, ensuring UI edits stay in lockstep with script values.

Dropdown Menu Template

The dropdown component renders both single- and multi-select pickers, keeping a MATLAB-bound property in view with editable labels and add/delete actions. It hides the stock controls and replaces them with custom popups, optional checkboxes, and an "add custom" flow that persists new options. For multi-select it summarizes selections ("All", count, or none) while exposing the full quoted list in the assignment preview; single-select mirrors the current choice. An edit/add mode lets users rename the `cfg` property, save via `matlabExecutor`, or remove the assignment entirely. Z-index management, focus handling, and inline validation keep the control usable in dense layouts.

Input Box Template

The input box presents a single-line field bound to a MATLAB property, with optional numeric mode, placeholder text, and a compact display of the exact assignment (quoted for strings, raw for numbers). Edit/add mode lets users rename the `cfg` key, preview the assignment, and save or delete it through `matlabExecutor` hooks; focus and z-ordering keep the control usable in dense layouts. Text changes emit live signals so downstream logic can react immediately, while a save icon commits the value and a trash icon removes the property when exposed in edit contexts.

Analysis Module Template

The Analysis Module Template is a reusable QML component providing a consistent interface for configuring and executing MATLAB-based analyses. It encapsulates complex functionality into a self-contained, configurable unit supporting any analysis type while maintaining visual and behavioral consistency. The template handles the complete lifecycle from parameter configuration through execution and visualization.

Core Architecture The component implements a collapsible panel design conserving screen space. Initially collapsed, showing only a header bar with the analysis name, it expands on click to reveal parameter controls and action buttons. This accordion-style pattern allows multiple

analysis modules on one page without overwhelming users, letting them focus on one analysis at a time while remaining aware of others. Height dynamically adjusts based on expansion state.

Parameter Management and Dynamic Rendering When instantiated, the template leverages the MATLAB parameter parser through the `matlabExecutor` object, which analyzes the corresponding MATLAB script and extracts all configurable parameters with their types, defaults, and ranges. This information returns as JSON, deserialized into the `dynamicParameters` property serving as the data model for interface generation.

A Repeater iterates over all parameters, delegating rendering to a DynamicParameterLoader that examines metadata and selects appropriate control types: range sliders for numeric ranges, step range sliders for stepped sequences, dropdowns for enumerations, input boxes for strings/numbers, and checkboxes for booleans. Each control receives configuration directly from parameter metadata, automatically matching interface constraints to MATLAB script requirements without manual synchronization.

Workflow Controls Two action buttons become visible when expanded:

Feature Extract Button: Primary action triggering analysis computation via a `buttonClicked` signal. Uses solid blue styling emphasizing its role. Always visible when expanded, allowing users to execute or re-execute analyses with current parameters.

Visualize Button: Secondary action appearing only when (1) the module is expanded AND (2) the expected output file exists in the working directory. The `checkOutputFile` function monitors output file existence through folder change events, automatically revealing visualization capabilities once results are available. This design eliminates user need to remember analysis execution status; the interface reflects data availability.

Visual hierarchy reinforces workflow: Visualize button appears above Feature Extract with white background and blue border indicating secondary role. Both implement hover detection with pointer cursor changes.

File Validation and Error Handling The `validateTargetFile` function verifies necessary data exists before execution. For analysis modules, this target is typically `data_ICApplied_clean.mat` from preprocessing. The validator iterates through folder contents with case-insensitive comparison, setting descriptive error messages if required files are missing. An inline error text component displays messages in red, positioned near relevant controls.

Advanced Features **Edit Mode**: When `editModeEnabled` is true, parameter controls expose additional functionality including renaming, syntax editing, and deletion. This transforms the template from consumption to configuration tool, enabling experienced users to customize analyses without leaving the application.

Integration: The template connects to parent page components and backend through property bindings and signal emissions. Properties like `currentFolder` and `folderContents` bind to shared application state tracking file browser location. Properties `moduleName`, `outputFileName`, and `visualizerFunction` parameterize each instance for its specific analysis type. Emitted signals `buttonClicked` and `visualizeClicked` trigger MATLAB executor calls with current parameters.

Extensibility: The template aliases the contentContainer's data property as default, allowing parent code to inject custom content into the expanded area, supporting future requirements like module-specific help text or specialized controls without template modifications.

Z-index Management: Explicit z-index values of 1000 ensure expanded modules and controls appear above collapsed modules in stacked scenarios.

The template abstracts configuration and execution complexity behind an intuitive expand-collapse-configure-execute pattern that works uniformly across all analysis types. Parameter discovery, interface generation, validation, state tracking, and backend communication are handled automatically, accelerating development of new modules while ensuring improvements benefit all analysis types simultaneously.

Classifier Template

The Classifier Template orchestrates the complete machine learning workflow for training, configuring, and testing deep learning classifiers across multiple analysis types and architec-

tures (EEGNet, EEG-Inception, Riemannian geometry-based approaches). Unlike the Analysis Module Template, it provides state management, persistent configuration storage, real-time logging, and testing capabilities within a unified interface.

Core Architecture The template uses the collapsible panel design pattern, expanding to reveal analysis selection controls, time window sliders, classifier-specific parameters, class selection menus, action buttons, and logging displays. Dynamic height adjustment accommodates varying parameter counts, ensuring smooth layout transitions. Key properties:

- `classifierName`: Identifies classifier architecture; determines configuration files and backend functions
- `selectedAnalysis`: Tracks chosen analysis type (ERP, spectral, time-frequency, connectivity, intertrial coherence)
- `configParameters`: JavaScript object containing hyperparameters (layer dimensions, regularization, epochs, batch sizes) loaded from JSON and rendered via input boxes with immediate persistence
- `analysisTimeSelections`: Dictionary mapping classifier-analysis pairs to user-selected time windows; persists across sessions and application restarts

Time Window Management Different analysis types require different temporal ranges due to distinct neuroscientific patterns. The `analysisTimeRanges` property stores four-element arrays [defaultStart, defaultEnd, minTime, maxTime] for each analysis:

- `ERP Analysis`: Default -0.2 to 1.0s; bounds -1.0 to 2.0s (baseline and post-stimulus event-related potentials)
- `Spectral Analysis`: Default 0 to 1.0s; bounds -0.5 to 3.0s (post-stimulus power changes)
- `Time-Frequency Analysis`: Default -0.2 to 1.0s; bounds -2.0 to 2.0s (extended baseline for spectral dynamics)

The `timeWindowProgrammaticUpdate` flag prevents circular update loops by distinguishing user-driven slider changes from programmatic updates, using Qt's `callLater` to reset the flag asynchronously. The `applyTimeWindowRange` function synchronizes both local properties and slider positions while managing persistence through the `savePersistedTimeSelections` method.

Analysis Selection and Configuration Loading When users select an analysis from the dropdown, `loadConfigurationForAnalysis` orchestrates a cascading sequence:

1. Reload persisted time selections from disk (critical for multi-user/multi-instance concurrent access)

2. Query classifier controller via `loadAvailableClasses` for distinct class labels in selected analysis's feature data

3. Request classifier-specific parameters through `getParamsForAnalysis` (returns JSON with all hyperparameters for this classifier-analysis combination)

4. Initialize time window bounds and slider positions based on saved selections or appropriate defaults

This ensures all interface elements remain synchronized with the selected analysis type, preventing stale parameter persistence from previous selections.

Dynamic Parameters and Class Selection A Repeater iterates over `configParameters` keys, instantiating InputBoxTemplate for each parameter. Type detection enables appropriate validation (numeric vs. textual). The `onValueChanged` handler implements immediate persistence: updates the in-memory object, invokes the controller's `saveConfiguration` method, and ensures configuration files always reflect interface state.

Class selection uses a split-button design with two sections: the main classify action and a right-side arrow toggle. The dropdown menu reveals checkboxes for each available class mapped to `selectedClasses` object entries. JavaScript immutability requires creating new

objects with copied values when toggling selections to trigger QML property change notifications.

Training Workflow and Logging The classify button validates input (analysis selected; at least one class chosen), then initializes training:

- Clears previous logs; sets `classifyExecuted` flag to true
- Populates initial log entries with timestamp-prefixed messages documenting configuration
- Emits `classifyClicked` signal carrying classifier name, analysis name, selected classes array, and complete `configParameters` object

A dedicated Window component displays a dark-themed console with monospaced text, automatically scrolling to show recent messages. The Connections component registers a `logReceived` handler that checks the `classifyExecuted` flag and appends timestamped messages from the Python backend. Title dynamically incorporates classifier name for context during concurrent operations. An "Open Console" button allows users to show the window after closing without restarting classification.

Weights Detection and Auto-Selection The `checkWeightsFileExists` function monitors folder contents for files matching the pattern **[classifierName]_[analysisKey]_weights_best.keras**, checking for substring presence to accommodate qualifiers. When detected, the `weightsFileExists` property enables interface elements depending on trained model availability, and `getWeightsFilePath` constructs the absolute path.

The `tryAutoSelectAnalysisFromWeights` function automatically expands the template and selects an analysis if the current directory contains matching weights files for any supported analysis, activating only when users have not manually selected an analysis. This streamlines workflows for resuming training sessions.

Time Range Integration and Testing The "Select Time Range" button invokes the MATLAB executor's `runSelectTimeRange` method, triggering a MATLAB script that loads feature

data, extracts temporal samples within the configured window bounds, and saves the windowed dataset. This enables experiments comparing classifier performance across different temporal windows.

A Test Classifier Window (currently disabled) provides a self-contained testing environment with properties for classifier name, selected analysis, weights file path, sample lists, and prediction results. The window connects to controller signals `onSamplesLoaded` and `onSingleSampleResult`, populates a dropdown with blind test samples (hiding ground truth labels), and displays predictions with visual hierarchy: predicted vs. actual labels in large boxes, correctness indicator (green for correct; red for incorrect), confidence percentage, and horizontal progress bars for class probability distributions. Dark-themed console at bottom displays test-specific logs with automatic scrolling.

Error Handling and Signal Communication An inline `errorTextItem` (red text, positioned above action buttons) displays contextual error messages without modal dialogs, such as ”Please select an analysis type first” or ”Please select at least one class”. This follows best practices by positioning feedback adjacent to error sources.

The template emits three signals:

- `classifyClicked`: Complete classification request (classifier name, analysis name, selected classes, parameters)
- `testClassifierClicked`: Testing request (classifier name, analysis name, weights file path; currently inactive)
- `classificationComplete`: Results folder path for post-training navigation or visualization workflows

Signal-based architecture enables loose coupling between template and parent page implementations.

Advanced Features **Default Property Alias**: Theoretically enables injection of custom content into the expanded area via `contentContainer`’s data property, accommodating future

classifier-specific elements (training progress curves, configuration wizards, documentation panels) without template modifications.

Z-index Management: Root template item (z-index 1000) and classify dropdown menu (z-index 1001) ensure proper visual stacking when expanded. Essential for scenarios with multiple classifier templates preventing dropdown portions from rendering behind higher templates.

The Classifier Template abstracts parameter loading, configuration persistence, class selection, weights detection, and execution logging behind a consistent, intuitive interface regardless of classifier architecture or analysis type, enabling users to efficiently configure and monitor classification workflows.

II.5.2 MATLAB Parameter Parser & Dynamic Parameter Loader

MATLAB Parameter Parser

The parser in `src/parser/matlab_parameter_parser.py` scans MATLAB scripts for `cfg.*` assignments, classifies each match, and normalizes the result into a typed dictionary. Dedicated regular expressions capture ranges, start–step–end triplets, scalars, strings (with boolean coercion), colon or bracket arrays, and cell arrays that become multi-select string options. Matches are processed in file order with a small priority scheme so overlapping expressions resolve predictably, while internal helpers such as `ft_paths` are skipped. Each parsed entry records its inferred type plus sensible defaults (e.g., latency values gain a millisecond unit). The `ModuleParameterMapper` ties user-facing module names to one or more MATLAB files so multi-file modules merge their parameters. `DropdownOptionStore` loads curated dropdown metadata from JSON, lets users add or remove options, and persists updated bounds when sliders expand their ranges. Finally, `create_ui_component` combines the parsed value and curated metadata into a UI-ready spec (component type, labels, defaults, and selection limits) that the QML layer can render directly.

Dynamic Parameter Loader

The loader bridges the parsed parameters to the interface. `src/parser/dynamic_parameter_loader.py` resolves module file paths, parses them, enriches each parameter with stored dropdown meta-

data, and emits a JSON map consumed by QML. On the front end, `features/analysis/qml/DynamicParam` picks the correct control (range slider, stepped slider, dropdown, input box, checkbox), binds change signals to a globally exposed `matlabExecutor`, and mirrors edits back into MATLAB scripts. Range sliders push numeric spans, dropdowns support single or multi-select with custom option persistence, and input or checkbox updates serialize to MATLAB assignments. Helper logic keeps dropdown icons aligned inside nested layouts, exposes getters and setters for restoring state, and honors an edit mode to toggle between interactive and read-only views. Together, the Python loader and QML component keep the GUI and MATLAB configuration synchronized without manual rewrites.

II.5.3 Matlab Executor

The MATLAB Executor serves as the central bridge between the Python-based graphical interface and the MATLAB computational backend, orchestrating the entire execution lifecycle of neuroscientific analyses. This component embodies one of the most technically challenging aspects of NeuroPAC's architecture, managing cross-language communication, asynchronous process execution, file system operations, and dynamic configuration management across multiple analysis modules. The implementation resides within the `matlab_executor.py` file and comprises approximately five thousand lines of carefully structured code that handle dozens of interconnected responsibilities.

Architectural Foundation and Core Responsibilities

The MATLAB Executor is implemented as a class named `MatlabExecutor` that inherits from Qt's `QObject` base class, establishing it as a full participant in Qt's signal-slot communication mechanism. This inheritance enables seamless integration with the QML-based user interface through reactive property bindings and asynchronous signal emissions. Upon instantiation, the executor initializes several critical internal states including the current data directory path, references to various MATLAB script locations, and instances of helper objects such as the parameter parser and preprocessing bridge. The constructor also discovers and stores the project root directory, enabling consistent path resolution across different deployment scenarios whether

the application runs in development mode or as a compiled executable through PyInstaller.

The class defines multiple PyQt signals that enable asynchronous communication with the user interface without blocking the main thread. The `outputChanged` signal delivers execution results to display panels, `configSaved` confirms successful parameter updates with detailed messages, `fileExplorerRefresh` triggers re-scanning of the file browser after data transformations, `processingFinished` notifies the interface when long-running operations complete, and `scriptFinished` returns results from background script executions. This signal architecture ensures that the interface remains responsive during computationally intensive MATLAB operations that might otherwise freeze the application.

Asynchronous MATLAB Execution Framework

One of the executor's most sophisticated features is its background thread management system for running MATLAB processes without blocking the user interface. The implementation employs a dedicated worker thread class called `MatlabWorkerThread` that executes MATLAB commands in isolated processes while the main application continues handling user interactions. When preprocessing or analysis operations are initiated, the executor spawns a worker thread configured with the MATLAB installation path, target script directory, and console visibility preferences. The worker constructs appropriate subprocess commands using either batch mode for headless execution or desktop mode when user interaction with the MATLAB console is desired.

The worker thread's execution logic handles numerous edge cases and error conditions. It normalizes file paths to use forward slashes for MATLAB compatibility regardless of the Windows-native backslash convention, wraps MATLAB commands in try-catch blocks to prevent silent failures, and implements comprehensive timeout mechanisms to prevent indefinitely hanging processes. When MATLAB completes execution, the worker emits a `finished` signal carrying a result dictionary that includes the return code, standard output, standard error, and any timeout indicators. The main executor receives this signal through its `_onMatlabFinished` callback method, which interprets the results, extracts relevant information such as the number of processed files, and generates appropriate user-facing messages before emitting them

through the `configSaved` signal.

This asynchronous architecture proves essential for operations like Independent Component Analysis that can require several minutes to complete. Without background threading, the entire application would freeze during processing, creating a perception of unresponsiveness or failure. The worker thread pattern ensures that users can continue interacting with the interface, viewing documentation, or preparing subsequent analysis steps while computationally intensive operations proceed in the background.

Configuration Persistence and File Manipulation

The executor implements an extensive system for persisting user-configured parameters to MATLAB script files, ensuring that interface changes directly modify the underlying analysis code. This bidirectional synchronization maintains consistency between what users see in the GUI and what MATLAB executes, eliminating common sources of confusion where interface settings appear ignored because they were never actually written to the scripts. The persistence system operates through a family of slot methods decorated with `@pyqtSlot` that accept parameters from QML interface components and translate them into MATLAB syntax.

When a user adjusts a parameter in the interface, the corresponding QML component invokes one of several specialized persistence methods depending on the parameter type. The `saveDropdownPropertyToMatlab` method handles selection-based parameters, formatting single values as quoted strings or multiple values as MATLAB cell arrays. The `saveRangeSliderPropertyToMa` method processes numeric ranges, generating either bracketed array syntax for discrete intervals or colon notation for continuous ranges depending on the parameter's semantic meaning. The `saveCheckboxPropertyToMatlab` method converts boolean states into MATLAB's yes-no string convention, with special handling for interdependent parameters like demean and baseline window that must be enabled or disabled together.

The actual file manipulation occurs through a sophisticated pattern-matching and replacement system implemented in helper methods. The `_replace_or_insert_matlab_assignment` method uses regular expressions to locate existing parameter assignments within MATLAB scripts, preserving comments and maintaining consistent formatting while updating only the

assignment value. If no existing assignment is found, the method intelligently determines an appropriate insertion point, typically placing new assignments before key function calls like `ft_preprocessing`. This approach ensures that dynamically added parameters integrate seamlessly with hand-written code rather than being appended haphazardly to file ends where they might be ignored or cause errors.

Special cases receive dedicated handling to manage complex interdependencies. The demean parameter in preprocessing, when disabled, must also comment out the baseline window specification since demeaning with a baseline window is meaningless. The `_save_demean_with_baseline` method implements this logic by coordinating modifications to both properties, using `_comment_matlab_assi` and `_uncomment_matlab_assignment` helpers to preserve the baseline window values in commented form rather than deleting them entirely. This preservation allows users to re-enable demeaning with their previously configured baseline without needing to remember and re-enter the values.

Dynamic Parameter Reading and Interface Initialization

Complementing the configuration writing system is an equally comprehensive parameter reading framework that initializes the interface with current values from MATLAB scripts when modules are loaded. This capability ensures that the GUI always reflects the actual state of the analysis configuration rather than defaulting to arbitrary initial values that might not match the script contents. The executor provides numerous slot methods prefixed with `getCurrent` that extract specific parameter values through targeted regular expression searches.

Methods like `getCurrentPrestim`, `getCurrentPoststim`, and `getCurrentEventvalue` parse preprocessing script contents to retrieve trial definition parameters. The `getCurrentBaselineWindow` method demonstrates sophisticated error handling by first searching for uncommented baseline window specifications and falling back to commented versions if no active specification exists, ensuring that users can see their baseline configuration even when demean is currently disabled. The `getCurrentChannels` method reads channel selection arrays from the preprocessing module, handling variations in whitespace and formatting that might arise from manual script editing.

These reading methods employ defensive programming practices that gracefully handle missing or malformed parameters. Each method wraps its file operations and parsing logic in exception handlers that log detailed error messages for debugging while returning sensible default values to prevent interface initialization failures. This robustness ensures that the application remains functional even when working with partially configured or manually modified scripts that might not conform to expected patterns.

The parameter reading system integrates with the dynamic parameter loader during module initialization. When a user selects an analysis module, the executor's `getModuleParameters` method orchestrates the discovery and extraction of all parameters from the module's associated MATLAB files. This method delegates file parsing to the `MatlabParameterParser` class, aggregates parameters from multiple script files when modules span several files, updates range slider bounds based on current values to ensure sliders can display and modify existing settings, and generates JSON-formatted component specifications that the QML interface uses to construct appropriate input controls.

Path Resolution and Cross-Platform Compatibility

Managing file paths correctly across Python, MATLAB, QML, and the Windows operating system presents significant challenges that the executor addresses through systematic normalization and conversion. The executor maintains multiple helper methods dedicated to path handling, ensuring that paths remain valid as they traverse different runtime environments. The `resource_path` function resolves relative paths to absolute paths that work both during development and in PyInstaller-packaged executables where resources are extracted to temporary directories.

When receiving paths from QML, the executor strips the `file:///` URL scheme prefix that Qt's file dialogs prepend, converts forward slashes to backslashes for Windows path operations, and normalizes multiple backslashes that might accumulate during conversions. Before passing paths to MATLAB, the executor performs the reverse transformation, converting backslashes to forward slashes since MATLAB interprets backslashes as escape characters in string literals. This bidirectional path normalization occurs in methods like `updateDataDirectory`

and updateFieldtripPath, ensuring consistent behavior regardless of how users select directories.

The executor also implements intelligent path resolution for locating MATLAB script files across the project structure. Methods like _get_preprocess_data_script_path, _get_timelock_script and similar functions maintain lists of candidate paths where scripts might reside, checking each location in order of likelihood and returning the first valid match. This approach accommodates different installation scenarios, allows for restructured project layouts during development, and provides fallback paths for PyInstaller-bundled resources that might be extracted to non-standard locations.

Custom Component Management

The executor implements a sophisticated system for managing dynamically created interface components that users add through the GUI, enabling extensibility beyond the predefined parameter set. This functionality supports custom dropdown menus and range sliders that users can create to expose additional MATLAB configuration options without requiring code modifications. The implementation involves direct manipulation of QML source files, inserting properly formatted component definitions that persist across application restarts.

When a user requests a custom dropdown through the interface, the addCustomDropdownOption method receives parameters specifying the dropdown's label, MATLAB property binding, multi-select behavior, and initial item lists. The method generates a complete QML component definition using the _build_custom_dropdown_snippet helper, which constructs a properly indented and escaped DropdownTemplate block with all necessary properties including persistence identifiers, label text, option arrays, and selection limits. The insertion logic locates a designated container element within the QML file using _locate_custom_container_bounds, finds the container's closing brace through brace-matching depth tracking, and splices the new component definition into the file content just before the closing brace.

Similar functionality exists for custom range sliders through methods like addCustomRangeSliderOption and _build_custom_range_slider_snippet. These methods generate RangeSliderTemplate definitions with numeric bounds, step sizes, units, and current values. The executor assigns

each custom component a unique identifier following a sequential numbering scheme, determined by scanning existing component IDs and incrementing from the maximum found index. This ensures that multiple custom components coexist without identifier collisions.

The executor also provides removal and modification operations for custom components. The `deleteCustomDropdown` and `deleteCustomRangeSlider` methods locate component blocks using regular expression searches, remove them from the QML content, and rewrite the file. Update operations employ a replace strategy, generating a new component definition with modified parameters and substituting it for the old block while preserving the component's identifier. These manipulation capabilities transform the interface from a static parameter display into a living configuration system that users can adapt to their specific analysis requirements.

Module-Specific Targeting and Script Selection

A critical aspect of the executor's design is its ability to route parameter updates to the correct MATLAB script files based on the active analysis module. Different modules employ different script files located in distinct directories, and parameters with identical names in different modules must modify different files to avoid cross-contamination of configurations. The executor implements a module-aware targeting system that maps module names to their corresponding script paths and ensures parameters reach their intended destinations.

The core of this system resides in conditional logic within persistence methods that examines the `module_name` parameter to determine appropriate target scripts. For instance, `saveDropdownPropertyToMatlab` contains branching logic that assigns the preprocessing data script when `module_name` equals "Preprocessing", the spectral analysis script for "Spectral Analysis", the time-frequency script for "Time-Frequency Analysis", and the timelock script for "ERP Analysis". Each branch constructs a list of target scripts, and subsequent code iterates through this list performing the same update operation on each file.

This architecture supports scenarios where a single parameter might legitimately need to appear in multiple scripts. For example, channel selection parameters could apply to both preprocessing and analysis stages, requiring updates to scripts in both modules. The executor

handles such cases by populating the target script list with multiple paths when appropriate, then iterating through all targets to ensure consistent configuration across the analysis pipeline.

The module targeting system also implements fallback logic for backward compatibility and error recovery. When an explicit module name is not provided, the executor attempts to infer the appropriate target based on the parameter name itself. Properties like `cfg.foi` and `cfg.taper` strongly suggest spectral analysis, while `cfg.toi` and `cfg.width` indicate time-frequency analysis. This inference allows legacy interface code that predates the explicit module naming convention to continue functioning correctly. However, the executor prints warning messages when relying on inference, encouraging developers to provide explicit module names for clarity and preventing ambiguous cases where inference might select incorrect targets.

Data Formatting and MATLAB Syntax Generation

Translating user interface values into valid MATLAB syntax requires careful attention to numerous formatting conventions and data type distinctions. The executor implements a family of formatting helper methods that generate syntactically correct MATLAB code from various Python data structures. These formatters handle the impedance mismatch between Python's dynamic typing and MATLAB's specific syntax requirements for different data types.

The `_format_matlab_assignment_value` method serves as the primary string and array formatter. It receives a list of values and a boolean indicating whether cell array format is required, then generates either a single quoted string, a numeric literal, a bracketed numeric array, or a cell array with curly braces depending on the input characteristics. Single values that parse as numbers are rendered as numeric literals without quotes, while non-numeric values receive single-quote wrapping with proper escaping of embedded quotes using MATLAB's double-quote escape convention. Multiple values are formatted as cell arrays with space-separated quoted elements, ensuring MATLAB interprets them as string cell arrays rather than character arrays.

Numeric range formatting employs specialized methods for different MATLAB range representations. The `_format_matlab_numeric_range` method generates two-element bracketed arrays for discrete ranges like frequency bands or time windows. The `_format_matlab_colon_range`

method produces colon notation for sequences specifying start, step, and end values such as time-of-interest vectors in time-frequency analysis. The formatter intelligently strips unnecessary trailing zeros and decimal points from floating-point representations, rendering ”5” instead of ”5.000000” for integer-valued floats while preserving meaningful decimal places for non-integer values.

Boolean parameters receive special handling through the checkbox persistence logic, which converts Python boolean values to MATLAB's string-based yes-no convention. This conversion recognizes that MATLAB configuration parameters typically expect `'yes'` or `'no'` strings rather than logical true-false values, reflecting FieldTrip's configuration style. The formatter wraps these strings in single quotes and inserts them into assignments that MATLAB's preprocessor can correctly interpret.

The formatting system also manages comment preservation during parameter updates. When replacing parameter assignments, the executor uses regular expressions that capture any trailing comments on the assignment line, preserving them in the updated line to maintain documentation and user notes embedded in the scripts. This attention to detail prevents the automated updates from destroying valuable human-readable information that helps users understand their configuration choices.

Classification Pipeline Integration

Beyond preprocessing and feature extraction, the executor provides specialized support for the classification module, handling data transformation and time window selection operations that bridge MATLAB feature extraction with Python machine learning frameworks. The classification integration relies heavily on the `PreprocessBridge` class, which the executor instantiates during initialization with the current data directory path. This bridge object abstracts the complexity of loading MATLAB-generated feature matrices and transforming them into NumPy arrays suitable for scikit-learn and TensorFlow models.

The `loadAndTransformData` slot method exemplifies this integration, accepting an analysis type specification and target model name from the classification interface. The method delegates to the preprocessing bridge's `load_and_transform` function, which loads appropri-

ate `.mat` files from the data directory, extracts relevant fields based on the analysis type such as ERP amplitudes or spectral power values, reshapes multidimensional arrays into feature vectors compatible with classifier input requirements, and returns a NumPy array ready for training. The executor wraps this process in comprehensive exception handling, capturing file-not-found errors when data has not been generated, key errors when analysis types are misspelled or data structures do not match expected formats, and generic exceptions for unexpected failures, returning descriptive JSON error messages to the interface for user feedback.

The time window selection functionality, implemented through `runSelectTimeRange` and its worker thread `_run_select_time_range_worker`, enables classifiers to focus on specific temporal regions of interest within trials. When invoked, the executor launches a background thread that constructs a MATLAB command calling the `select_time_range.m` script with classifier name, analysis type, data path, and a JSON configuration file specifying window boundaries. The MATLAB script opens in desktop mode, allowing users to interactively inspect the data and refine their selections if needed. Upon completion, the script writes time-windowed data to a new output folder with a `_timeranged` suffix, and the executor emits signals to refresh the file browser so users can immediately locate and utilize the transformed data.

This classification support demonstrates the executor's role as an integration hub, coordinating multiple technologies and frameworks to present users with a unified workflow despite the underlying heterogeneity of computational tools.

Error Handling and User Feedback

Throughout its extensive functionality, the executor implements comprehensive error handling and user feedback mechanisms that transform raw exceptions and failures into actionable information. Nearly every public slot method wraps its core logic in try-except blocks that catch anticipated exceptions, log detailed diagnostic information to the console for debugging, and emit user-friendly messages through the `configSaved` signal that appear in the interface's status display. This approach ensures that errors do not crash the application or leave it in undefined states, instead providing users with clear descriptions of what went wrong and, when

possible, suggestions for resolution.

Error messages distinguish between different failure modes with appropriate specificity. File-not-found errors clearly state which file is missing and suggest prerequisite operations that should be completed first, such as running preprocessing before attempting ICA component visualization. Parse errors in parameter reading methods indicate which parameter could not be extracted and fall back to reasonable defaults, allowing the interface to initialize in a partially configured state rather than failing entirely. Subprocess execution failures report MATLAB return codes and include both standard output and standard error from the MATLAB process, giving users visibility into MATLAB's own diagnostic messages.

The executor also implements graceful degradation strategies for non-critical failures. When custom component manipulations fail due to unexpected QML file structures, the executor logs warnings but does not prevent normal operation since custom components are optional enhancements rather than essential features. When parameter writes fail for one target script in a multi-target update, the executor continues updating remaining targets and reports a summary of successes and failures rather than aborting the entire operation after the first error.

This defensive programming philosophy, combined with extensive logging and transparent error reporting, transforms the executor from a brittle integration point that could easily fail into a robust coordination layer that maintains operational stability even when confronting unexpected conditions.

State Management and Configuration Synchronization

The executor maintains several pieces of internal state that track the application's configuration and execution status, ensuring consistency across asynchronous operations and multiple interface interactions. The `_current_data_dir` property stores the actively selected data directory, initialized by reading the preprocessing script at startup and updated whenever users select new folders through the file browser. This cached value enables quick reference without repeatedly parsing scripts, and the executor propagates updates to dependent objects like the preprocessing bridge whenever the data directory changes.

The `_option_store` instance, a `DropdownOptionStore` object, maintains persistent stor-

age of custom dropdown options and range slider bounds across application sessions. This store serializes to a JSON configuration file, and the executor invokes its save method whenever custom options are added or range limits are updated. During module parameter discovery, the executor queries the option store to retrieve saved configurations for each parameter, merging them with current values from MATLAB scripts to produce complete component specifications. This integration ensures that dynamically added options persist across application restarts and remain synchronized with script-based configurations.

Thread tracking through properties like `_worker_thread` and `_select_time_thread` prevents concurrent execution of operations that should not overlap. Before launching new background processes, the executor checks whether existing threads are still alive and emits warning messages if users attempt to start duplicate operations. This safeguard prevents resource exhaustion from accumulating subprocess instances and avoids race conditions where multiple threads might simultaneously modify the same files.

The executor's state management embodies a consistent philosophy of single-source-of-truth, where MATLAB script files serve as the authoritative configuration source, the Python executor maintains runtime state derived from those files, and interface components reflect the executor's state through reactive property bindings. This unidirectional data flow pattern, inspired by modern frontend frameworks, eliminates many categories of synchronization bugs and makes the application's behavior more predictable and maintainable.

II.5.4 File Browser Window

The file browser serves as the primary interface for navigating the file system, selecting datasets, and labeling experimental data. It bridges the QML frontend with a Python backend using a reactive architecture: a `FileBrowser` class (inheriting from `QObject`) manages file operations, while `FileBrowserUI` QML component maintains signal-slot connections that ensure the interface always reflects backend state.

Backend Implementation and Path Normalization The Python `FileBrowser` class maintains three internal states exposed through Qt properties: current folder path, folder contents

list, and processed file data in RAM. Property changes emit `pyqtSignal` events that trigger QML updates without explicit method calls, eliminating synchronization issues.

The `loadFolder` method handles cross-platform path complexity through defensive normalization:

1. Receives paths from Qt's file dialog (format: `file:///C:/Users/...`)
2. Strips `file:///` prefix and removes double backslashes
3. Converts forward slashes to backslashes for Windows `os` module compatibility
4. Uses `os.listdir()` to enumerate items; `os.path.isdir()` determines type
5. Prefixes files with document symbol, directories with folder symbol
6. Emits `folderContentsChanged` signal with formatted items; catches exceptions (inaccessible paths, permission errors) and reports to user

This robust approach ensures consistent path handling regardless of source (file dialogs, MATLAB scripts, user input).

Frontend Architecture and Visual Layout The `FileBrowserUI` component uses Qt's `Connections` element to register handlers for backend signals, exemplifying the model-view-controller pattern: Python backend (model/logic), QML component (view/controller). The layout occupies the left 20

- **Upper region**: File explorer displaying folder contents with scrollable list view
- **Lower region**: Data labeling interface for annotating datasets

Header bar (spanning both regions) displays the current folder path with two buttons:

- `Refresh` button: Reloads folder contents without changing directories
- `Folder Selection` button: Opens native file picker

Responsive height computation allocates screen space dynamically, maintaining minimum heights to ensure usability. Computed properties calculate available space and distribute fairly between explorer and labeling sections.

File Interaction and Type-Specific Handling The file explorer renders items as interactive elements with hover effects. Hover state displays light blue background; MATLAB files (`.mat`) gain underline styling. Color coding distinguishes file types:

- `.mat` files: Blue text (indicating interactivity)
- ICA component files (filename contains "ICA"): Green text with distinctive icon

Click handling implements type-specific behavior:

- **data.mat files**: Emit `fileMatClicked` signal; parent component populates labeling interface with file contents
- **ICA component files**: Delegate to MATLAB executor, launching FieldTrip's interactive `ft_databrowser` for component inspection and rejection
- **Other files**: Simple notification signal (no further action)

Data Labeling Interface When a `data.mat` file is selected, the system queries the Python backend through `matlabExecutor` to retrieve dataset names (MATLAB `.mat` files can contain multiple named arrays and structures). The backend:

- Returns dataset name list
- Loads previously saved labels from persistent storage
- Populates list model with dataset-label pairs

The interface presents pairs as input fields bound to the list model through QML property binding, automatically persisting user-entered label changes. When users press enter or move focus away from a field, labels save to Python storage, establishing permanent dataset-semantic associations.

Advanced Features **Dual-Pane Design**: File explorer and labeling interface in stacked layout keeps frequently accessed controls visible while providing specialized functionality below. This thoughtful UX maintains immediate access to core operations while organizing secondary features.

External Refresh: The `refreshFromExternal()` function provides a public entry point for parent components to reload folder contents without requiring user interaction. Enables seamless workflows where classification results or analysis outputs automatically appear in the browser immediately after computation, without manual refresh clicks.

The file browser abstracts file system complexity and persistent storage behind a clean interface, enabling users to efficiently organize and annotate datasets for subsequent analysis modules.

II.5.5 Preprocessing-Specific

ICA Browser

The ICA Browser enables inspection and rejection of independent components extracted from EEG data, bridging automated decomposition with human judgment for identifying physiological artifacts (eye blinks, muscle activity, cardiac signals).

Data Loading and Variable Detection The browser loads preprocessed ICA data from a specified MATLAB file path, performing intelligent variable detection by examining all non-metadata variables. The detection algorithm prioritizes `data_ICApplied` (conventional ICA decomposition container) but flexibly identifies ICA data structures through characteristic FieldTrip properties: channel labels, trial matrices, temporal information arrays. This structural validation enables operation on diverse preprocessing outputs without rigid naming conventions.

Sensor-Space Data Handling The system attempts to locate a companion `data.mat` file containing raw sensor-level measurements. If found, loads directly. Otherwise, implements a fallback mechanism reconstructing sensor-space data from ICA components via matrix multiplication: applies topographical transformation matrices to project independent components

back to original electrode space. Reconstruction preserves metadata (sampling rates, trial information, recording headers) by traversing FieldTrip's configuration history chain.

Interactive Review and Batch Processing The browser processes subjects sequentially, presenting FieldTrip's `ft_databrowser` in component view mode. This displays each component's temporal evolution alongside its topographical scalp distribution, enabling artifact identification through time-series and spatial signatures. Rejection decisions accumulate in a structured array maintaining subject-component correspondence.

After interactive review, an automated pipeline applies rejection decisions in batches (10 subjects at a time for memory efficiency). For each subject, reconstructs channel signals from only retained components, filtering artifact-related neural activity while preserving cognitive signals. Batch processing with incremental disk persistence protects against data loss and allows interrupted sessions to resume from last successful batch.

Stimulus-Specific Decomposition and Outputs Following artifact removal, the browser segments each cleaned dataset into three condition categories (target, standard, novelty stimuli) by examining trial information codes. Prepares data for condition-specific analyses (ERP averaging, spectral power comparisons).

Three primary outputs:

- **Cleaned data**: Complete temporal/spatial structure with artifacts removed
- **Decomposed data**: Experimental conditions separated into discrete structures
- **Rejection records**: Audit trail documenting which components were rejected per subject for quality control

All outputs persist to disk in MATLAB's compressed format and export to workspace for immediate validation.

Reject IC Button & `ft_databrowser` Synchronization

Integrating NeuroPAC's ICA workflow with FieldTrip's `ft_databrowser` required a strategic modification approach extending FieldTrip capabilities while maintaining compatibility with

future updates and minimizing disruption to users' existing installations.

Architecture: Minimally Invasive Modification Rather than forking FieldTrip entirely (creating maintenance burden and breaking shared installations), the solution adopts a minimally invasive approach modifying only ICA component rejection functionality. Two parallel versions exist:

- **Reference version**: Stored in NeuroPAC at `features/preprocessing/matlab/ft_databrowser_mo` containing all ICA rejection modifications
- **User's installation version**: Located in user's FieldTrip directory, receives modifications through automated synchronization

Core Modification: Reject IC Button The modification introduces a "reject IC" button in `ft_databrowser`'s right column control panel using normalized coordinates for resolution independence. Button instantiation via `uicontrol` specifies parent figure handle, pushbutton style, and unique user data identifier 'rc' linking to callback handler.

When clicked, the interface enters interactive selection mode. The callback implements position-to-component mapping by analyzing spatial layout and determining which timecourse is nearest to click coordinates. The algorithm differs by view mode:

- **Butterfly view**: All components overlay in single axis
- **Vertical/component view**: Each element occupies dedicated spatial region; directly queries figure's timecourse objects using `findobj` with 'timecourse' tag, exposing y-data through graphics handles. Identifies temporal sample nearest to horizontal click coordinate, then determines component by finding which timecourse has nearest y-value at that time sample. This reverse approach (rendered graphics $\rightarrow$ data indices) proves robust by implicitly accounting for all FieldTrip scaling/transformation operations

Rejection Data Structure and Visual Feedback Once a component is identified, the system extracts component number/label from the data structure's channel label array. Accesses

`cfg.current_subject_index` passed by `browse_ICA.m`, which iterates through subjects sequentially. Maintains cell array `opt.reject_ICs` (one cell per subject, initialized with zeros):

- Single rejection: Cell contains numeric scalar
- Multiple rejections: Transitions to nested cell array with numeric values

This hybrid structure balances memory efficiency with query simplicity.

Visual feedback highlights the selected timecourse in red with "(REJECTED)" label at click location, persisting until user selects different component or navigates to different trial/subject. Console prints diagnostic messages (component name, number, subject index) creating audit trail.

When databrowser closes, a cleanup callback packages state into output configuration, adding `cfg.rejected_ICs` field to standard FieldTrip outputs. This minimal semantic extension maintains backward compatibility.

Synchronization Mechanism The Python script `quick_sync.py` propagates modifications from NeuroPAC's reference version to user's FieldTrip installation. Implements intelligent update system:

1. Loads preprocessing configuration to extract user's FieldTrip installation path
2. Compares modification timestamps of reference and installation versions
3. If reference newer: Synchronizes; if installation current/newer: Exits
4. Performs targeted cleanup removing standalone testing blocks (path setup commands)
5. Writes modified content to user's `ft_databrowser.m` via direct replacement
6. Includes comprehensive error handling (permission problems, missing files, unexpected structures)

Timestamp-based approach provides efficient change detection without expensive file content comparison.

Integrates as optional preliminary step users invoke through menu option before ICA analysis. Transparent instructions explain installation modification and recommend backing up original `ft_databrowser.m`. Users declining synchronization retain preprocessing features but lack integrated component rejection.

Communication Flow and Result Collection `browse_ICA.m` constructs configuration structures with standard FieldTrip parameters (layout specifications, view mode) plus NeuroPAC-specific context (`cfg.total_subjects`, `cfg.current_subject_index`). After databrowser returns, extracts rejection information from `cfg.rejected_ICs` field, consolidating into master rejection array with defensive extraction logic handling various data structure possibilities (missing fields, empty arrays, nested cells, inconsistent indexing).

The consolidated array drives artifact removal: iteratively invokes FieldTrip's component rejection function per subject with appropriate rejection indices. Separation between interactive selection and batch processing allows continuous subject review without waiting for computations, enables decision revision before final processing, and facilitates batch optimization.

The architecture reflects key software engineering principles: minimal modification reduces upstream divergence; timestamp-based synchronization provides automatic updating; transparent user notification respects autonomy while streamlining setup.

II.5.6 Analysis-Specific

Analysis Visualizer

Shared interaction model All visualizers follow a consistent MATLAB GUI pattern to keep analysis exploration predictable. Each tool: (i) normalizes inputs into a subjects-by-condition struct array (target/standard/novelty) to avoid dissimilar-structure errors; (ii) stores state in the figure via `guidata` (current subject, selected channels or pairs, axis overrides); (iii) provides Previous/Next navigation to iterate subjects; (iv) offers checkbox panels to select channels (or channel pairs) with an "all" toggle; (v) exposes `xlim`/`ylim` overrides through edit boxes that immediately re-render plots; and (vi) hides the figure until UI controls are fully built to avoid flicker. All UIs enforce defensive handling of empty or malformed inputs and clamp selections

to valid indices.

FieldTrip dependency handling The time–frequency and coherence visualizers attempt to auto-locate FieldTrip (e.g., checking `ft_singleplotTFR`/`ft_connectivityplot` availability) and mirror the fallback search logic used in preprocessing scripts. When available, FieldTrip plotting routines are invoked with interaction disabled (e.g., `cfg.interactive = 'no'`) so that NeuroPAC's own UI remains authoritative.

ERP visualizer (time-domain averages) Displays per-channel event-related potentials in a channel-by-condition grid using `avg` waveforms. Typical use: rapidly compare latency/amplitude differences across target, standard, and novelty conditions with optional manual axis limits. Uses the recorded channel labels when present, falling back to autogenerated labels otherwise.

Spectral visualizer (frequency-domain power) Plots power spectral density curves derived from `fourierspctrm` for each selected channel. Supports subject paging and channel selection identical to ERP, with optional frequency/Power axis overrides to focus on bands of interest. Generates default channel labels when none are provided.

Time–frequency visualizer (TFR surfaces) Renders time–frequency power maps per condition via FieldTrip's `ft_singleplotTFR`. Requires at least one channel selection; otherwise it suppresses plotting with a user message. Preserves provided `layout` when present, and applies optional time/frequency window overrides. Input normalization accepts either a subjects-by-3 matrix or a vector of structs with {target, standard, novelty} fields, copying only known fields to avoid structure mismatch.

Channel-wise coherence visualizer Targets connectivity outputs (coherence) across channel pairs. Builds a pair list from `labelcmb` or square coherence matrices, then plots time–frequency coherence per selected pair across the three conditions. Provides pair-level checkboxes (two-column layout when many pairs exist) plus shared axis overrides. Extracts pair-specific slices into FieldTrip-compatible TFR structs for plotting.

Inter-trial coherence visualizer Shows per-channel ITPC/ITLC maps, switchable via a coherence mode dropdown (phase or linear). Otherwise mirrors the ERP-style navigation and channel selection. Uses `itpc` or `itlc` fields when present, with graceful skips for missing data and optional axis limits to highlight time–frequency regions.

II.5.7 Classification-Specific

Preprocess Bridge

Purpose and role The Python module `preprocess_bridge.py` bridges MATLAB analysis outputs and Python deep-learning models. It discovers and loads FieldTrip-derived `.mat` files (ERP, time–frequency, spectral, connectivity, inter-trial coherence), normalizes their structures, pads variable shapes, and returns tensors plus labels suitable for model input.

Core data structures `MatlabStructElement` wraps MATLAB structs so they behave like Python dicts and attributes while preserving keys, shapes, and simple metadata. `PreprocessBridge` holds resolved data paths, filename mappings per analysis type, expected condition order (target, standard, novelty), and flexible group label mappings (e.g., HC/CTL vs. PD variants) sourced from a shared `labels` list.

Loading pipeline `load_and_transform()` cleans incoming paths (stripping `file:///` prefixes, normalizing slashes), picks the right `.mat` filename, then loads with `scipy.io.loadmat`; MATLAB v7.3 files automatically fall back to `h5py`. It selects the correct struct key per analysis (e.g., `ERP_data` with `avg`, `timefreq_data` with `powspctrm`), iterates subjects in struct arrays, derives dataset names from `cfg.dataset` when available, and attaches group labels from `labels`. For each condition it retrieves the primary data field with fallbacks (trial averaging when needed), converts complex arrays to magnitudes, handles spectral/connectivity-specific averaging, cleans NaN/Inf values, and records condition and subject indices.

Shape normalization and transforms After extraction, samples are squeezed, dimension-aligned by padding to the max observed shape, and stacked into X. Labels y include condition, group, subject id, and dataset name. `_apply_transform()` reshapes ERP data for

EEGNet/EEG-Inception (adds singleton spatial dimension) and preserves 4D tensors for time–frequency and inter-trial coherence when targeting EEG-Inception, while passing through Riemannian targets untouched.

Filtering and selection `filter_by_selected_classes()` supports class-specific downselection (e.g., only PD-target) by parsing class strings into group/condition pairs, normalizing group labels (numeric or textual), and returning filtered samples plus kept indices.

MATLAB v7.3 handling Helper methods (`_load_mat_v73`, `_convert_mat73_struct_array`, `_convert_condition_struct`) dereference HDF5 references, extract only relevant data fields, and wrap them in `MatlabStructElement`, ensuring v7.3 files remain interchangeable with earlier `loadmat` outputs.

Controller

Console

Inference

Main

Model

Test

II.6 Research Design and Procedures

II.7 Data Description

II.7.1 Parkinson's

II.8 Instrumentation

II.9 Limitations

II.10 Summary

Chapter III

Results

Chapter IV

Conclusion and Future Work

IV.1 Conclusion

IV.2 Future Work

Appendix A

Assessment Activity

(Content...)

Appendix B

Assessment Rubric

(Content...)